

**SJSU CMPE 172**

# CMPE 172 – MILESTONE 2 REPORT

Core Implementation: Booking & Concurrency

- **Name: Canhui Huang**
- **Student ID: 017553913**
- **Date: 10/08/2026**
- **Chosen scenario: Tutoring appointments**
- **GitHub repository (milestone tag): https://github.com/canhuiHuang/tutoringappointment/tree/milestone-2**
- **Code-walkthrough video link: https://drive.google.com/file/d/1I2d9oYg6jLG8ODiSDpyP3o9HqlT7AYmF**

## Overview

This application allows students to book tutoring appointments and tutors to provide availability. Milestone 2 extends the original database and layered application with login, role-based access, booking, cancellation, appointment history, and concurrency protection. Customers can filter available slots by tutor, service, and date, browse a weekly calendar, confirm a booking in a popup, view their appointments, and cancel their own future appointments. Providers can create and remove availability for their own services and view appointments booked with them.

The Web interface uses React.ts. The backend uses Java, Spring Boot, and JDBC. PostgreSQL stores the application data. The database identifies customers as STUDENT and providers as TUTOR.

## Booking flow

BookingPopup → frontend API function → BookingController → BookingService → BookingRepository → JdbcTemplate → PostgreSQL

## Frontend-to-backend flow

The React frontend communicates with Spring Boot through JSON APIs.

The frontend sends the selected slot and service IDs:

```javascript
  await api.post("/api/customer/appointments", {
    slotId: slot.id,
  });
```

Spring's DispatcherServlet routes the request, and AccessConfig checks the user's session and role. The controller obtains the student ID from the server session and calls the service:

The frontend sends the selected slot and service IDs:

```java
  long studentId = AccessConfig.account(request).id();
```

```
    long appointmentId =
        bookingService.book(studentId, input.slotId());
```

The service applies business rules, and the repository executes parameterized SQL. After the transaction commits, the controller returns HTTP 201 with the appointment ID. React displays confirmation and refreshes availability. Exceptions are handled globally and returned as JSON messages with appropriate HTTP status codes.

**Concurrency**

The main race condition occurs when two students attempt to book the same slot simultaneously. Without concurrency protection, both requests could read the slot as available and create duplicate bookings.

The application uses pessimistic row locking with PostgreSQL. Before checking availability, the repository locks the selected slot:

SQL

```
    SELECT a.*, s.duration
    FROM availability_slots a
    JOIN services s ON a.service_id = s.id
    WHERE a.id = ?
    FOR UPDATE OF a;
```

The first transaction obtains the lock, while the second waits. After the first commits, the second sees the updated BOOKED status and receives HTTP 409.

The chosen isolation level is READ_COMMITTED. It prevents dirty reads, while the explicit row lock protects the same-slot booking operation. This avoids requiring SERIALIZABLE isolation for every booking.

A unique database constraint provides additional protection:

SQL

```
    active_slot_id BIGINT UNIQUE
```

**Transaction design**

The booking operation uses Spring's @Transactional annotation with READ_COMMITTED isolation:

Java

```
 @Transactional(isolation = Isolation.READ_COMMITTED)
public long book(long studentId, long slotId, long serviceId) {
    // Lock and validate the slot before these writes.
    long appointmentId =
        bookingRepository.insertBooking(studentId, slotId);
    bookingRepository.slotStatus(slotId, "BOOKED");
    return appointmentId;
}
```

Appointment insertion and the slot status update belong to the same transaction. Both changes commit together; an

unchecked exception rolls back both, preventing a partially completed booking. The slot's row lock remains held until the transaction finishes.

Cancellation also runs within a transaction. It marks the appointment CANCELLED, clears its active slot reference, and restores the slot to AVAILABLE together. This preserves appointment history while allowing the future slot to be booked again.

**Role-based access**

The application uses server-side sessions and two roles: STUDENT for customers and TUTOR for providers. Login retrieves the user's credentials and role through JDBC. Passwords are stored as BCrypt hashes and verified using:

```java
boolean validPassword =
    passwordEncoder.matches(password, credentials.hash());
```

After successful authentication, the controller stores the account in the session:

```java
request.getSession().setAttribute("account", user);
```

AccessConfig checks protected endpoints on the backend. For example, customer endpoints require the student role:

```java
if (path.startsWith("/api/customer/")
        && !account(request).role().equals("STUDENT")) {
    throw new BookingException(403, "Customer role required.");
}
```

Provider endpoints similarly require TUTOR. Requests without authentication receive HTTP 401, and wrong-role requests receive HTTP 403.

The service also checks ownership. Students cannot cancel another student's appointment:

```java
if (initialBooking.studentId() != studentId) {
    throw new BookingException(
        403, "You may only cancel your own appointments.");
}
```

Providers can create slots only for services belonging to them. The repository verifies this relationship through SQL:

```sql
SELECT duration
FROM services
```

```
  WHERE id = ? AND provider_id = ?;
```

Provider slot removal also checks ownership and permits removal only when the slot is available. These backend checks enforce permissions independently of what the React interface displays.